Assignment No - 1

AIM: Implement binary search tree and perform the following operations:

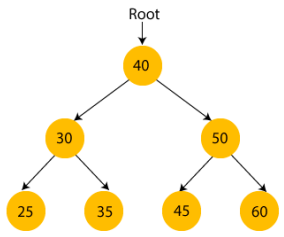
a. Insert
b. Delete
c. Display

OBJECTIVE:

- To understand the concept of binary search tree.
- To understand the representation of binary search tree.
- To understand how to create, insert, delete and display binary search tree.

THEORY:

In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.



Node Structure(in JAVA):

```
class Node{

        int key;

        Node leftChild, rightChild;

}
```

ALGORITHM:

1). Insertion Operation:

- Check the value to be inserted (say X) with the value of the current node (say val) we are in:
    - If X is less than val move to the left subtree.
    - Otherwise, move to the right subtree.
- Once the leaf node is reached, insert X to its right or left based on the relation between X and the leaf node's value.

2). Deletion Operation:

- If the root is NULL, then return root (Base case)
- If the key is less than the root's value, then set root->left = deleteNode(root->left, key)
- If the key is greater than the root's value, then set root->right = deleteNode(root->right, key)
- Else check
    - If the root is a leaf node then return null
    - else if it has only the left child, then return the left child
    - else if it has only the right child, then return the right child
    - else set the value of root as of its inorder successor and recur to delete the node with the value of the inorder successor
- Return

3). Inorder Traversal

- Step 1: Repeat Steps 2 to 4 while TREE != NULL
- Step 2: INORDER(TREE -> LEFT)
- Step 3: Write TREE -> DATA
- Step 4: INORDER(TREE -> RIGHT)

  [END OF LOOP]

- Step 5: END

CODE:

```java
import java.util.*;

public class BinarySearchTree {
    //Node Creation
    class NodeBT{
     int data;
     NodeBT LeftChild;
     NodeBT RightChild;

     NodeBT(int d){
         this.data = d;
         this.LeftChild = null;
         this.RightChild = null;
     }
    }

    public static NodeBT root = null;    //root node of the tree

    //Insert Operation
    public void InsertNode(int d) {
     root = insertRec(root, d);
     System.out.println("Inserted Successfully");
    }

    public NodeBT insertRec(NodeBT Root, int d) {
     NodeBT newNode = new NodeBT(d);
     if (Root == null) {
         Root = newNode;
         return Root;
     }

         if (d < Root.data) {    //data less than root
             Root.LeftChild = insertRec(Root.LeftChild, d);
         }
         else if ( d > Root.data) {    //data greater than root
             Root.RightChild = insertRec(Root.RightChild, d);
             //return root;
         }


     return Root;
    }

    //Traversals

    //1). Inorder traversal
    public void inOrder(NodeBT Root) {
     inOrderRec(Root);
    }
```

```java
public void inOrderRec(NodeBT Root) {

     if (Root != null) {
         inOrderRec(Root.LeftChild);
         System.out.print(Root.data + " ");
         inOrderRec(Root.RightChild);
     }
}

//delete node
void deleteNode(int d){
    root = deleteRec(root, d);

}

NodeBT deleteRec(NodeBT Root, int d){
    if (Root == null){
        return Root;
    }

    if (d < Root.data){
        Root.LeftChild = deleteRec(Root.LeftChild, d);
    }
    else if (d > Root.data){
        Root.RightChild = deleteRec(Root.RightChild, d);
    }
    else{
        if (Root.LeftChild == null){
            return Root.RightChild;
        }
        else if (Root.RightChild == null){
            return Root.LeftChild;
        }
        Root.data = minValue (Root.RightChild);
        Root.RightChild = deleteRec(Root.RightChild, Root.data);
    }
    return Root;
}

int minValue(NodeBT Root){
    int minv = Root.data;
    while (Root.LeftChild != null){
        minv = Root.LeftChild.data;
        Root = Root.LeftChild;
    }
    return minv;
}
```

```java
    //Searching

    boolean searchNode(int d){
        root = searchRec(root, d);
        if (root != null){
            return true;
        }
        else{
            return false;
        }
    }

    NodeBT searchRec(NodeBT Root, int d){
        if (Root == null || Root.data == d){
            return Root;
        }
        if (Root.data > d){
            return searchRec(Root.LeftChild, d);
        }
        return searchRec(Root.RightChild, d);
    }

    public static void main(String[] args) {
     // TODO Auto-generated method stub
     System.out.println("Binary Search Tree");
     BinarySearchTree obj = new BinarySearchTree();
     //8, 10, 12, 14, 33, 1, 9, 8, 3
     obj.InsertNode(8);
     obj.InsertNode(10);
     obj.InsertNode(12);
     obj.InsertNode(14);
     obj.InsertNode(33);
     obj.InsertNode(1);
     obj.InsertNode(9);
     obj.InsertNode(3);
     obj.inOrder(root);
     System.out.println();
     obj.inOrder(root);
     boolean flag = obj.searchNode(14);
     System.out.println(flag);
    }
}
```

INPUT/OUTPUT:

```
Binary Search Tree
Inserted Successfully
Inserted Successfully
Inserted Successfully
Inserted Successfully
Inserted Successfully
Inserted Successfully
Inserted Successfully
Inserted Successfully
1 3 8 9 10 12 14 33
1 3 8 9 10 12 14 33 true
```

CONCLUSION:

We learnt to implement Binary search tree using JAVA programming language. Also, we applied the algorithms to perform insertion, deletion and traversal operations on the binary search tree.